

 Software Training and Placement Center

DotNet Full Stack Developer Course

 HTML5, CSS3, Bootstrap

 JavaScript, TypeScript

 Angular

 C#, SQL Server

 ASP.NET Core, Web API

 Microservices, EF Core

Comprehensive 90-Day Program

What You Will Achieve

- ✓ Master Core C# & SQL
- ✓ Build RESTful APIs with ASP.NET Core
- ✓ Develop Dynamic Web UIs (Angular/React)
- ✓ Implement Full-Stack Real-Time Apps
- ✓ Ready for Job Interviews & Placements

System Requirements

OS: Windows 10/11 (64-bit) or macOS
RAM: 8GB+ (16GB Rec)
Storage: 256GB SSD+
CPU: Intel i5 / Ryzen 5+

Course Overview & Objectives

Master Full Stack Development with .NET ecosystem: From Frontend UI to Backend APIs & Database Management

What is Full Stack?

A Full Stack Developer handles both **Frontend** (User Interface) and **Backend** (Server Logic) development, plus database management. This course covers the complete .NET ecosystem.

90-Day Plan

Week 1-3 Frontend: HTML5, CSS3, Bootstrap

Week 4-6 JavaScript, TypeScript & Angular

Week 7-9 C# Core & SQL Database

Week 10-12 .NET Core API & Placement

What You Will Achieve

- ✓ Develop real-time RESTful APIs using ASP.NET Core & Entity Framework
- ✓ Build dynamic, responsive web apps with HTML, CSS, JS & Angular
- ✓ Master C# OOP concepts, SQL queries & database management
- ✓ Implement security with JWT, OAuth2 & microservices architecture
- ✓ Deploy full-stack applications & prepare for technical interviews
- ✓ Create portfolio projects & be placement-ready

Technology Landscape

Software Domains & Key Players

 Frontend

User Interface & Experience

React

Angular

Vue

Meta

Google

Apple

 Backend

Server Logic & APIs

.NET

Node.js

Java

Microsoft

Netflix

Amazon

 Data Science

Analysis & AI Models

Python

R

TensorFlow

Google AI

OpenAI

DeepMind

 Cloud & DevOps

Infrastructure & Deployment

AWS

Azure

Docker

AWS

Microsoft

GitLab

 Fresher Path

Start with fundamentals. Build strong foundation in programming, web basics, and database concepts. Focus on problem-solving and small projects.

1. HTML/CSS

2. JavaScript

3. C#/.NET

4. SQL

5. Projects

 Experienced Path

Deep dive into advanced architecture, design patterns, microservices, and cloud integration. Focus on system design and scalability.

1. Adv .NET

2. Microservices

3. Cloud

4. DevOps

5. Leadership

 Job Application Portals

 LinkedIn Jobs

 Indeed

 AngelList (Wellfound)

 Company Career Pages

 Naukri.com

 Glassdoor

 HackerRank Jobs

 Referral Networks

How the Internet Works

Part 1: From URL to Data Transfer

What Happens When You Click a URL?

- 1 DNS Lookup:** Browser checks Domain Name System to translate "www.google.com" to IP address (e.g., 142.250.190.46)
- 2 TCP Connection:** Browser establishes connection with server using 3-way handshake
- 3 HTTP Request:** Browser sends GET request to server for webpage content
- 4 Server Processing:** Server receives request, processes it, and prepares response
- 5 HTTP Response:** Server sends HTML, CSS, JS files back to browser
- 6 Rendering:** Browser interprets files and displays webpage to user

How Routers Work

Routers direct data packets between networks. They use routing tables to determine the best path for data transmission.

Mobile Towers

Convert signals between mobile devices and fiber networks

WiFi Routers

Connect local devices to ISP via wired/wireless connection

Internet Balance & Data Usage

ISP tracks data usage by counting packets sent/received. Each packet contains metadata (headers) and actual data (payload).

How Balance is Deducted:

1. Request sent → Upload data counted
2. Response received → Download data counted
3. Total = Upload + Download (in MB/GB)

File Size Calculation

Unit	Size	Example
Byte	8 bits	1 character
Kilobyte (KB)	1,024 Bytes	Short email
Megabyte (MB)	1,024 KB	MP3 song
Gigabyte (GB)	1,024 MB	HD movie
Terabyte (TB)	1,024 GB	500 hours of video

How the Internet Works

Part 2: Data Storage, File Types & Infrastructure



Binary Data Storage

Computers store all data as **0s and 1s** (binary). Each 0 or 1 is a "bit". 8 bits = 1 Byte.

```
01001000 01100101 01101100 01101100 01101111
= "Hello" in binary
```

Colors are stored as RGB values (0-255 each). Example: **Red** = 11111111 00000000 00000000



Data Centers & Satellites

Data Centers: Physical facilities housing servers, storage systems, and network equipment that power the internet.

Satellites: Provide internet connectivity to remote areas by relaying signals between ground stations and users.



File Types & Formats

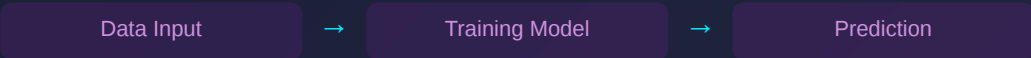
Every file type has a specific structure for storing data as byte streams.

.mp4 Video	.mp3 Audio	.pdf Document
.jpg Image	.html Web Page	.exe Executable



AI Systems & Environment

How AI Works:



 **Environmental Considerations**

- Data centers consume massive energy for cooling & power
- Green initiatives: Renewable energy, efficient cooling systems
- AI training requires significant computational resources

Computer Science **Ecosystem**



Core Theory



DLD



DM



FLAT



Compiler Design



DAA



Systems Foundation



Operating Systems



Computer Networks



DBMS



Computer Organization



Development & Application



Coding Languages



Full Stack Development



Web Technologies



Mobile Dev



Integrated Knowledge

Theory + Systems = Applications

Execution Map: Java & C#

How Programming Languages Execute & Process Requests

⚙️ Flow 1: Execution Pipeline

<> Source Code

Developer writes `.java` or `.cs` code. Human-readable instructions using language syntax.

Programming

🔑 Compilers

`javac` translates Java to Bytecode. `csc` translates C# to IL. Checks syntax and semantics.

Compiler Design

⚙️ JVM / CLR

Virtual Machine provides runtime environment. `JIT` compiles bytecode to machine code.

OS / Architecture

🔧 RAM Execution

OS allocates `Heap` for objects, `Stack` for methods. CPU executes machine code.

OS & Memory

↔️ Flow 2: Request-Response Cycle

📄 User Input

User sends request via `HTTP`/Browser or App interface.

UI/UX

☁️ Request Process

Server receives request via `Network Protocols` (TCP/IP). Routes to handler.

Computer Networks

Σ DSA/Logic

Executes business logic using `Data Structures & Algorithms`.

DSA

🗄️ DB Connection

Connects to `DBMS` via `JDBC/ADO.NET`. Executes SQL queries. Sends response back.

DBMS

Technology Stack & Required Software

Tech Stack



.NET Core

HTML

HTML5



CSS3



JavaScript



TypeScript



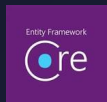
Angular



React



Azure Cloud



EF Core



SQL Server



C#



Gen AI / LLMs

Required Software

Visual Studio Code

Lightweight, powerful code editor for web and cloud development.

 code.visualstudio.com/download

Visual Studio 2022

Full-featured IDE for .NET and C# development.

 visualstudio.microsoft.com/downloads

Node.js (LTS)

JavaScript runtime for running Angular/React and package management.

 nodejs.org/en/download

.NET SDK 8.0

Core runtime, libraries, and compiler for building .NET apps.

 dotnet.microsoft.com/download

HTML & HTML5

Structure and present web content with multimedia, geolocation, and advanced APIs.

12

Modules

8

HTML5 APIs



Web Basics

Introduction, Tags, Attributes, Formatting

HTML4



Semantic Layout

Header, Nav, Section, Article, Footer, Aside

HTML5



Forms & Inputs

Validation, Date/Time, Color, Range, File Upload

HTML5



Canvas & SVG

2D Graphics, Drawing API, Vector Images

HTML5



Media Elements

Audio & Video Tags, Streaming, Embedding

HTML5



Web Storage

LocalStorage, SessionStorage, JSON Data

HTML5



Geolocation

GPS, IP Address, Mapping, Reverse Geocoding

HTML5



Structuring

Lists, Tables, Images, Hyperlinks, Grouping

HTML4

CSS & CSS3

Styling the web with colors, layouts,
formatting, and responsive transformations.

25

Modules

12+

CSS3 Features



Selectors & Combinators

Attribute, Pseudo, Child, ID, Class, General Sibling

Core



Box Model

Margin, Border, Padding, Content, Box-Sizing

Core



Layout Systems

Flexbox, CSS Grid, Positioning, Floats

Modern



Responsiveness

Media Queries, Viewport Units, Breakpoints

CSS3



Transitions & Transforms

2D/3D Transforms, Timing Functions, Rotation, Scale

CSS3



Animations

Keyframes, Animation Properties, Sequencing

CSS3



Typography

Fonts, Text Effects, Decoration, Shadow, Spacing

Core



Visual Effects

Gradients, Shadows, Border Radius, Opacity, Filters

CSS3

Bootstrap

Powerful, extensible frontend toolkit with prebuilt grid systems and components for responsive mobile-first sites.

7

Modules

20+

Components



Grid System

12-Column Grid, Auto-layout, Responsiveness, Nested Rows

Core



UI Components

Buttons, Cards, Modals, Carousel, Alerts, Badges, Tooltips

UI



Forms & Validation

Input Groups, Floating Labels, Select, Range, Validation

Forms



Columns & Layout

Vertical/Horizontal Alignment, Wrapping, Reordering, Offsetting

Layout



Navigation

Navbar, Navs, Tabs, Breadcrumbs, Pagination

Navigation



Content & Icons

Typography, Images, Figures, Tables, Bootstrap Icons

Content



Responsiveness

Breakpoints, Containers, Fluid Layouts, Mobile-First

RWD



Utilities & Helpers

Spacing, Display, Flex, Text, Border, Shadows

Utilities

Java Script

Lightweight, dynamic programming language with first-class functions for interactive web applications.

12

Modules

ES6+

Modern JS



Core Fundamentals

Variables, Data Types, Console API, Operators, Type Conversion

Basics



Control Flow

Conditions (if/else), Loops (for, while, do...while), Break/Continue

Logic



Strings & Arrays

String Methods, Array Operations, Iteration, Real-time Usage

Data Structures



Objects & Functions

Object Operations, Arrow Functions, Callbacks, Higher-Order Functions

Advanced



DOM Manipulation

Selectors, Traversing, Creating/Appending Nodes, Styling

Web API



Browser Events

Mouse, Keyboard, Form Events, Event Handling, Delegation

Interaction



Web Storage

localStorage, sessionStorage, Cookies, JSON Data

Storage



Async Internals

Event Loop, Call Stack, Task Queue, SetTimeout/SetInterval

Core Concepts

TypeScript

A syntactic superset of JavaScript adding static typing for better tooling and safety in enterprise applications.

6

Modules

ES6+

Standard



Getting Started

Installation, Setup, Basic Syntax, Hello World, ts-node

Setup



Why TypeScript

Readability, Tooling, Safety, Enterprise Adoption

Benefits

Core Types

Tuple, Boolean, Intersection, Number, String, Enum, Any, Void, Arrays

Types



Classes & Objects

Abstract Classes, Inheritance, Constructors, Accessors

OOP



Generics

Generic Interfaces, Classes, Functions, Constraints, Type Parameters

Advanced



ES6 Features

let/const, Arrow Functions, Promises, Classes, Modules, Destructuring

Modern JS

Angular

Robust framework for building scalable Single Page Applications (SPAs) using TypeScript and modern web standards.

16

Modules

v17

Latest



Architecture & Basics

Framework Overview, Folder Structure, Bootstrapping, Decorators

Core



Components & Modules

@Component, @NgModule, Data Binding, Lifecycle Hooks

Fundamental



Services & DI

@Injectable, Dependency Injection, HttpClient, Providers

Services



Routing & Navigation

RouterModule, Lazy Loading, Parameterized Routing, Guards

Navigation



RxJS & Observables

Operators, Subjects, Promises, Async Data Handling

Reactive



Forms & Validation

Template-driven, Reactive Forms, Custom Validators

User Input



Authentication & Security

JWT Auth, Interceptors, Route Guards, Role-based Access

Security



UI Libraries & Project

Angular Material, External Libraries, Real-time Project

UI/UX

C# Programming

Fast, secure, object-oriented language integral to the .NET ecosystem for web, mobile, and enterprise apps.

19

Modules

.NET 8

Latest



Introduction & Setup

Visual Studio, First Program, C# History, .NET Runtime

Basics



Core Fundamentals

Data Types, Variables, Operators, Flow Control, I/O Streams

Foundation



OOP Concepts

Classes, Objects, Inheritance, Polymorphism, Abstraction

OOP



Exception Handling

Try-Catch-Finally, Custom Exceptions, Throwing Exceptions

Error Handling



Collections & Generics

List, Dictionary, HashSet, Queue, Stack, Generic Classes

Data Structures



File I/O & Streams

StreamReader, StreamWriter, FileStream, File Operations

File System



Advanced C#

Delegates, Events, Lambda, LINQ, Async/Await

Advanced



Memory Management

Garbage Collection, IDisposable, Unmanaged Resources

Performance

SQL

Standard language for communicating with relational databases to manage and retrieve data efficiently.

2

Modules

MSSQL

Server



SQL Basics

Overview, Table Creation, DDL, DML, DQL, TCL, DCL, Primary/Foreign Keys

Fundamentals



SQL Clauses

SELECT, WHERE, GROUP BY, HAVING, ORDER BY, DISTINCT, Aliases

Queries



Joins & Unions

Inner/Outer Joins, Self Join, Cross Join, Union, Union All

Relationships



Operators & Functions

AND/OR/NOT, LIKE, Wildcards, IN, BETWEEN, Aggregate Functions

Operations



Constraints

NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, DEFAULT

Data Integrity



Views & Indexes

Creating Views, Updatable Views, Clustered/Non-Clustered Indexes

Optimization



Stored Procedures

Creating Procedures, Parameters, Variables, Control Flow, Error Handling

Programming



Advanced SQL

Transactions, Locks, Triggers, Cursors, Dynamic SQL, Performance Tuning

Advanced

.NET Core & Web API

Build production-grade, standalone RESTful microservices and web applications faster with ASP.NET Core.

Part 1

Core & Data

.NET 8

Framework



.NET Core Overview

Project Structure, .csproj Model, Templates, Internal Working

Core



ADO.NET

SqlConnection, SqlCommand, CRUD Operations, SqlDataAdapter

Data Access



Dependency Injection

IoC, Service Lifetimes, Constructor/Property Injection, Options Pattern

Architecture



ASP.NET Core

Middleware, Routing, Controllers, Layered Architecture, Services

Framework



Entity Framework Core

DbContext, Migrations, LINQ, Repository Pattern, Seed Data

ORM



Web API Development

RESTful Routing, Swagger, JSON, Model Binding, Validation

API



C# Service Configuration

Service Registration, Attributes, Primary Services, Qualifiers

Config



SQL Server Integration

Connection Strings, EF Core with MSSQL, Database Setup

Database

.NET Core & Web API

Advanced concepts for securing applications and building scalable microservices architecture.

Part 2

Security & Micro

OWASP

Top 10



ASP.NET Core Security

Authentication, Authorization, OWASP Top 10, Security Filters

Security



JWT Authentication

Token Generation, Validation, Middleware, Token Expiration

Auth



OAuth2 & CSRF

OAuth2 Providers, Third-Party Login, CSRF Protection, Cookies

Protection



User Management

ASP.NET Core Identity, User Tables, Password Hashing, BCrypt

Users



Microservices

Architecture, Service Registry, HTTP Clients, Inter-Service Communication

Architecture



API Gateway

Ocelot Gateway, Routing, Load Balancing, Reverse Proxy

Gateway



Service Discovery

Consul, Dynamic Registration, Service Location, Health Checks

Discovery



Cloud Native

Cloud Principles, Containerization, Scalability, Resilience

Cloud

Placement Preparation

Bridging the gap between technical skills and landing your dream job with comprehensive career preparation.

5

Key Areas

1:1

Mock Interviews



Communication Skills

Verbal & Written Skills, Professional Communication, Presentation Techniques

Soft Skills



Arithmetic & Logic

Aptitude, Logical Reasoning, Problem-Solving, Numerical Ability

Aptitude



Technical Interview Prep

Core Concepts, Algorithms, Data Structures, System Design Questions

Technical



HR Interview Prep

Behavioral Questions, Company Research, Salary Negotiation

HR Round



Mock Interviews

1-on-1 with Head Staff, Feedback Sessions, Performance Improvement

Practice



Resume Building

ATS-Friendly Formats, Portfolio Presentation, GitHub Profile

Profile

Your Journey Starts Now

Course Completed!

You've mastered HTML5, CSS3, JavaScript, TypeScript, Angular, C#, SQL, and .NET Core Web API. You are now a Full Stack Developer.

"The only way to do great work is to love what you do."

- Steve Jobs

Next Steps

- 1 Build Your Portfolio**
Create 3-5 projects showcasing full-stack skills
- 2 Master GitHub**
Maintain active contributions and clean documentation
- 3 Prepare for Interviews**
Practice DSA, system design, and behavioral questions
- 4 Apply Strategically**
Target companies aligned with your tech stack
- 5 Never Stop Learning**
Stay updated with latest technologies and frameworks